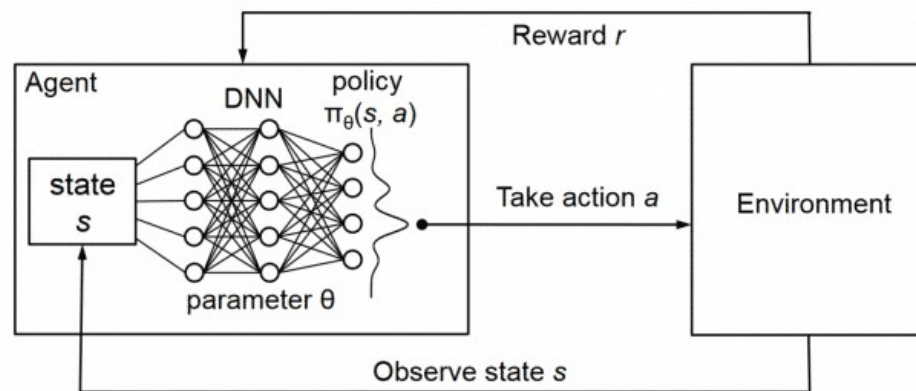
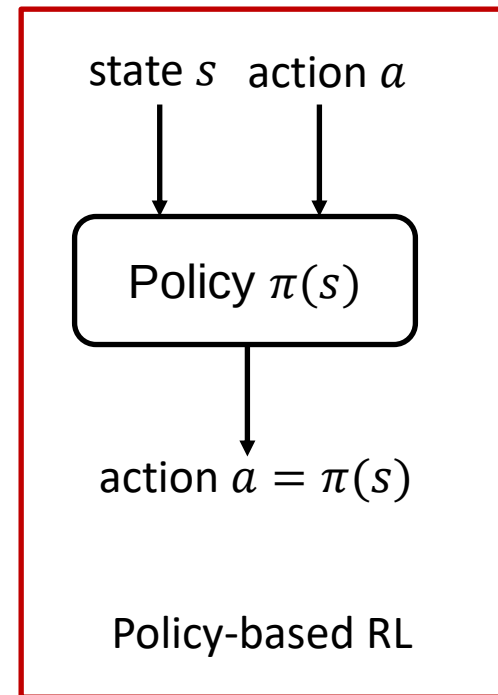
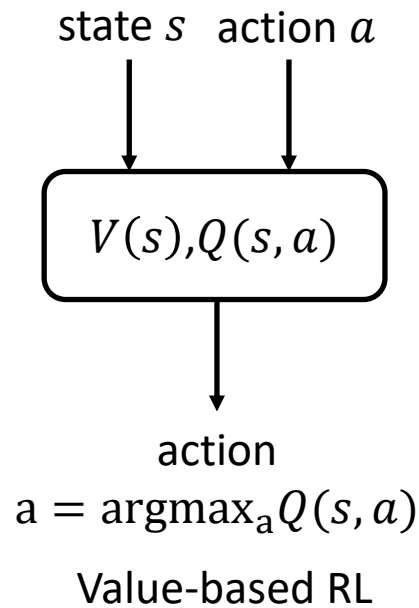
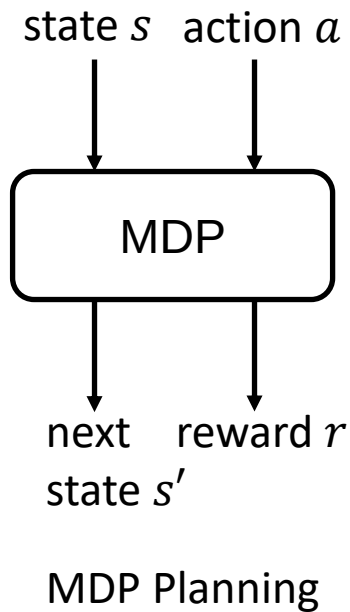


L9.3 Policy-based RL



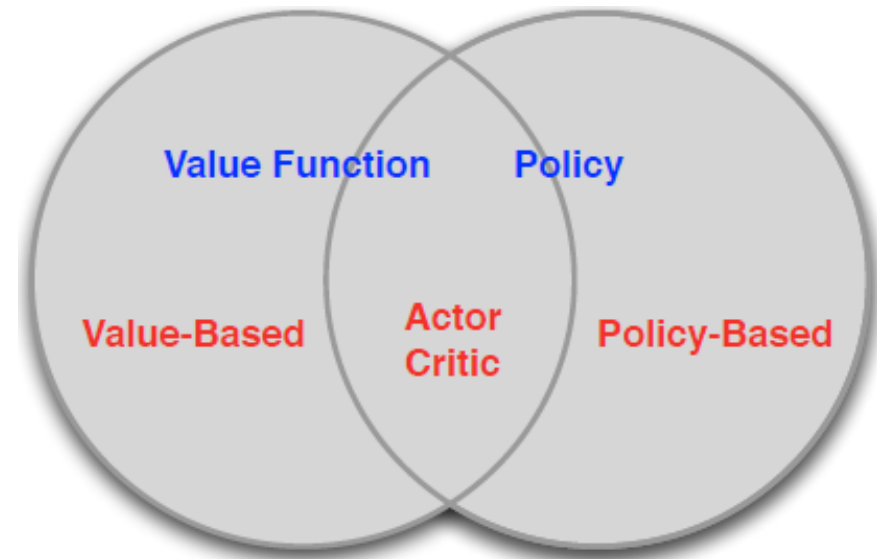
Zonghua Gu 2023

Policy-based RL



CH13 Policy Gradient Methods

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy

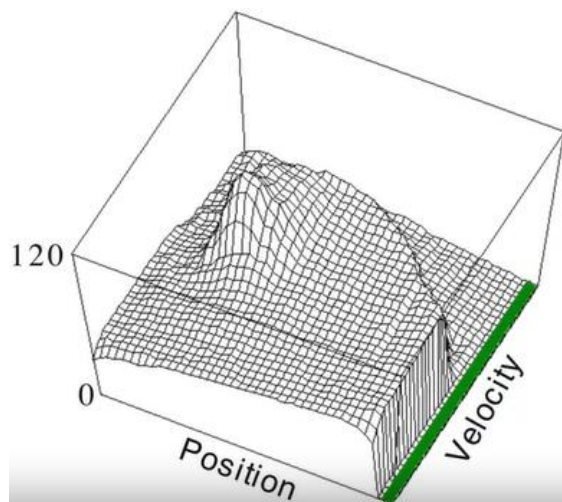
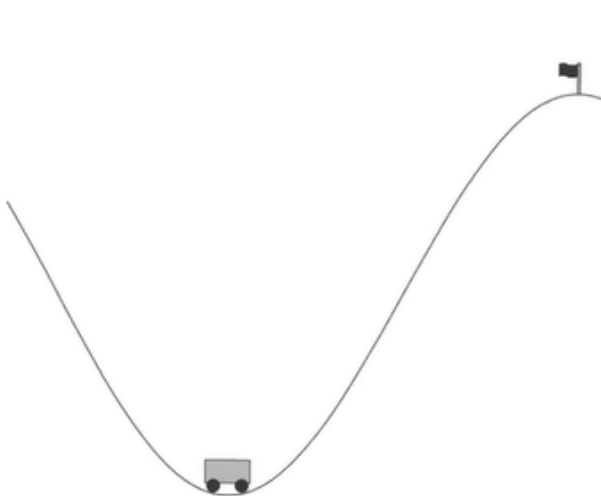


Policy-based RL Pros and Cons

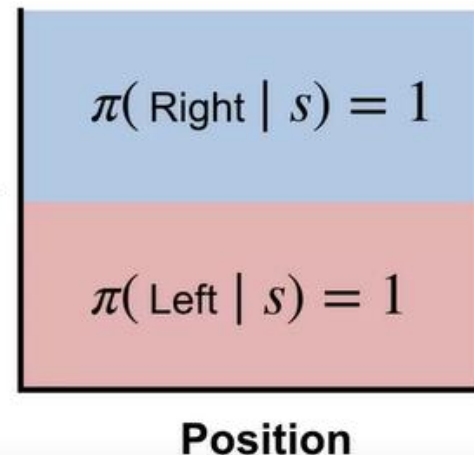
- Pros:
 - Effective in high-dimensional or continuous action space.
 - Value-based RL is only applicable to discrete action space; inefficient to discretize continuous actions for high-dim action space, as taking $\operatorname{argmax}_a Q(s, a)$ may be expensive.
 - Can learn stochastic policies
 - Value-based RL learns a near-deterministic policy (greedy or ϵ -greedy).
 - Policy typically converges faster than value functions (more sample efficient).
- Disadvantages:
 - Typically converges to a local rather than global optimum.
 - Evaluating a policy is typically inefficient and high variance.

Mountain Car Example

- An under-powered car situated in a valley wants to drive up a steep hill to reach the goal at the top of the rightmost hill. Due to gravity, the car cannot simply accelerate up the steep slope. It must learn to leverage potential energy by driving up the opposite hill before the car is able to make it to the goal.
- Middle: a complex value function
- Right: a simple policy that works well: accelerate in the direction of current velocity.

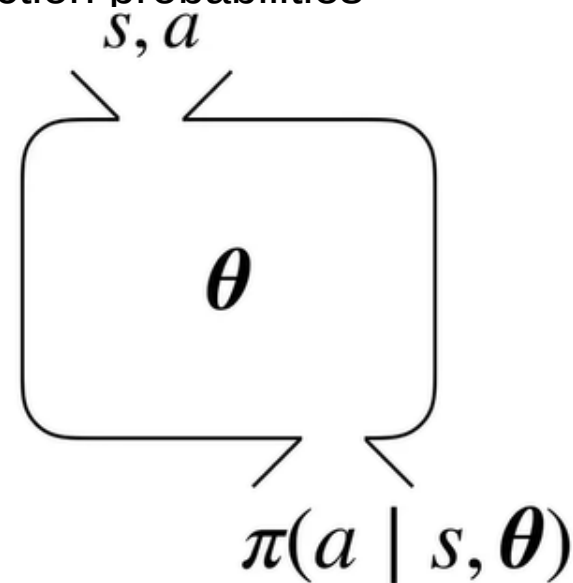
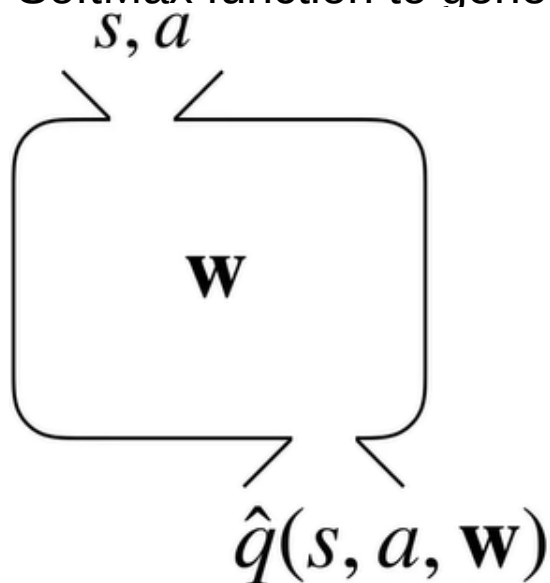


moving
right
Velocity
moving
left



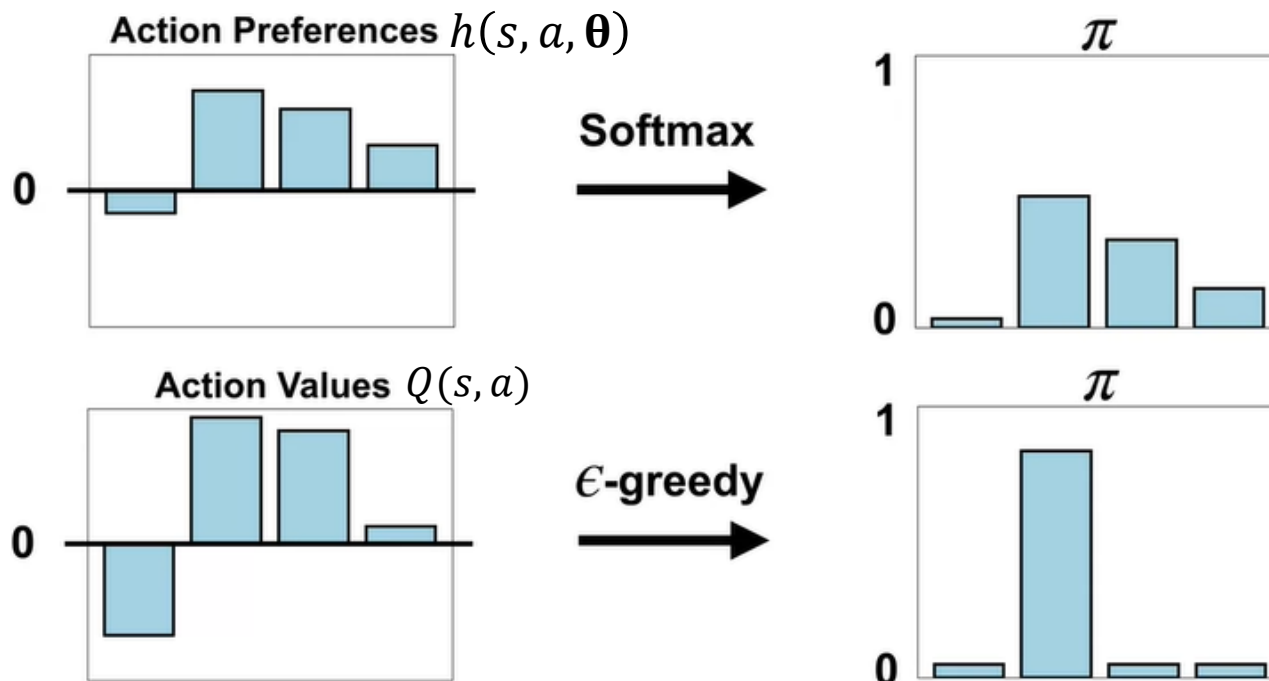
Function Approximation for Action Value Function vs. Policy

- Value-based RL learns a function approximation for action value function $\hat{q}(s, a, \mathbf{w})$.
 - Deterministic policy $a = \operatorname{argmax} \hat{q}(s, a, \mathbf{w})$ (may be ϵ -greedy during training)
- Policy-based RL learns a function approximation for stochastic policy $\pi(a|s, \boldsymbol{\theta})$:
 - Probability that action a is taken in state s , with parameter $\boldsymbol{\theta}$. The actual action taken is sampled from the probability distribution $A \sim \pi(a|s, \boldsymbol{\theta})$
 - Probability must be non-negative: $\pi(a|s, \boldsymbol{\theta}) \geq 0, \forall a \in \mathcal{A} \wedge \forall s \in \mathcal{S}$
 - Probabilities must sum to 1: $\sum_{a \in \mathcal{A}} \pi(a|s, \boldsymbol{\theta}) = 1, \forall s \in \mathcal{S}$
 - Use SoftMax function to generate the action probabilities



SoftMax vs. ϵ -greedy for Discrete Actions

- SoftMax policy: $\pi(a|s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_{a' \in \mathcal{A}} e^{h(s,a',\theta)}}$
 - $h(s, a, \theta)$ is action preference, which may be a linear function $\theta^T \mathbf{x}(s, a)$, or the logit from the SoftMax layer of a DNN
 - A bad action with very negative $h(s, a, \theta)$ will be very unlikely to be selected
 - Action probabilities change smoothly as a function of $h(s, a, \theta)$
- ϵ -greedy: select the greedy action $\arg\max_a Q(s, a)$ with prob $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}$
 - No distinction between policies that are not the optimal one; A bad action with very low $Q(s, a)$ will be selected with equal prob as all other non-optimal policies
 - Action probabilities may change dramatically for an arbitrarily small change in $Q(s, a)$, if that change results in a different optimal action

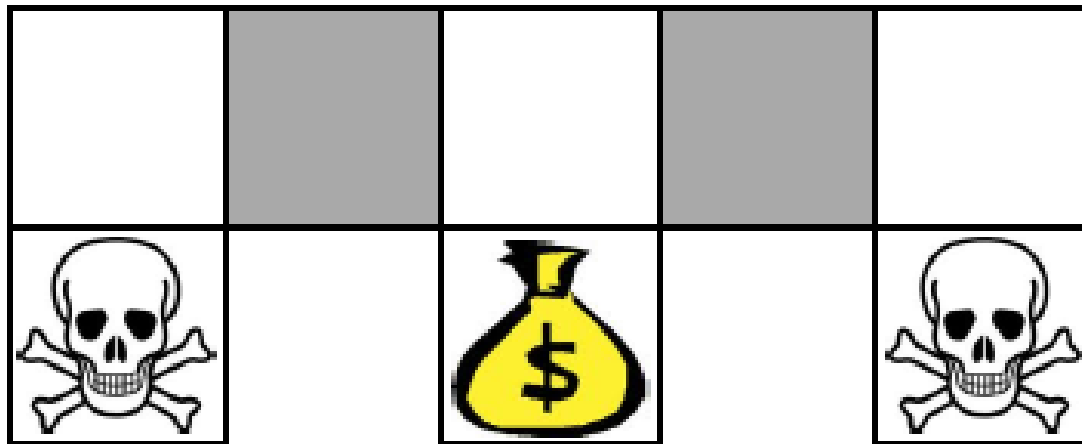


Stochastic Policy vs. Deterministic Policy

- Example 1: two-player game of rock-paper-scissors
 - Scissors beat paper; paper beats rock; rock beats scissors
 - For iterated game, a deterministic policy is easily exploited by the opponent; a uniform random policy is optimal, and achieves Nash equilibrium
- Example 2: Aliased Grid World (POMDP)

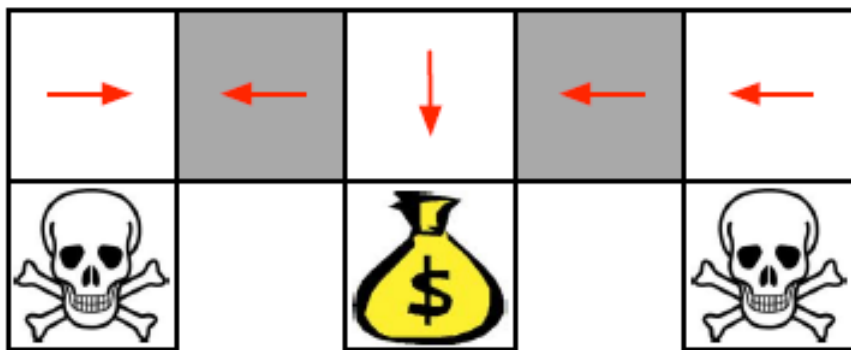
Aliased Grid World (POMDP)

- Env has 3 terminal states: 1 with high positive reward and 2 with high negative reward. It is a Partially Observable MDP (POMDP): Agent cannot observe its position directly; it can only observe features of the following form (for all directions $d_1, d_2, \dots \in (N, E, S, W)$):
 - $\phi(s) = \mathbf{1}(\text{wall to } d_1, d_2 \dots)$ (it can detect walls, e.g., w. Radar or Lidar)
 - $\mathbf{1}(x)$ is indicator function: $\mathbf{1}(x) = 1$ if $x = \text{true}$
 - Agent cannot differentiate between the 2 grey states, as both states have a wall to the north
- Value-based RL learns a deterministic policy: $a = \operatorname{argmax} \hat{q}(s, a, \mathbf{w}) = \operatorname{argmax} f(\phi(s), \mathbf{w})$
- Policy-based RL learns a stochastic policy: $\pi(a|s, \boldsymbol{\theta}) = g(\phi(s), \boldsymbol{\theta})$

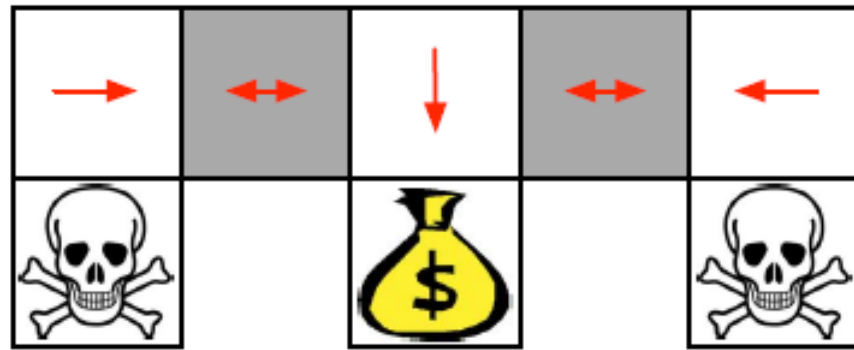


Aliased Grid World (POMDP)

- Left: an optimal deterministic policy:
 - Either move W in both grey states, or move E in both grey states
 - Either way, agent can get stuck and never reach the goal state
- Right: an optimal stochastic policy:
 - Randomly move E or W in grey states:
 $\pi(\text{move } E | \text{wall to N and S}, \theta) = \pi(\text{move } W | \text{wall to N and S}, \theta) = 0.5$
 - Agent will likely reach the goal state quickly



Opt. deterministic policy



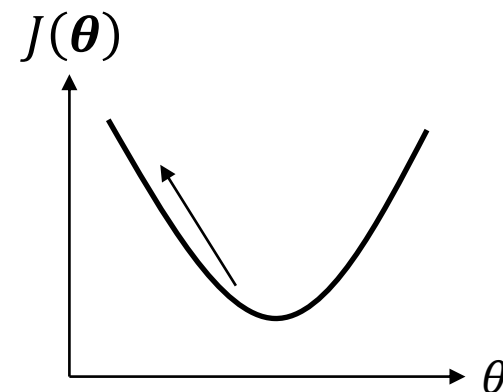
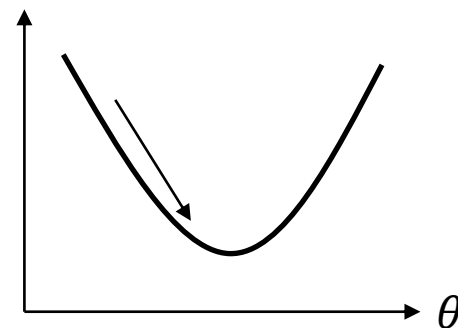
Opt. stochastic policy

Optimization Objective

- We consider the episodic case, and would like to optimize the expected value of the start state of each episode, with start state s_0 :
 - $J(\boldsymbol{\theta}) \doteq v_{\pi}(s_0)$
 - where $v_{\pi}(s_0)$ is the true value function for policy π , parametrized by $\boldsymbol{\theta}$: $\pi(a|s, \boldsymbol{\theta})$

Model Training in Supervised Learning vs. Policy Gradient in RL

- SL: to solve $\min_{\theta} \mathbb{E}_{(x,y) \sim D} \text{Loss}(x, y; \theta)$ for model $\mathbb{E}_{(x,y) \sim D} \text{Loss}(x, y; \theta)$
training: gradient **descent** $\theta \leftarrow \theta - \alpha \nabla_{\theta} \text{Loss}(x, y; \theta)$
 - Update **model params** θ by following the gradient **downhill**, in order to **decrease** $\text{Loss}(x, y; \theta)$. (α is the Learning Rate)
- RL: to solve $\max_{\theta} J(\theta)$ in Policy Gradient: gradient **ascent** $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$
 - Update **policy model params** θ by following the gradient **uphill**, in order to **increase** $J(\theta)$
- (We use ∇ as shorthand for ∇_{θ})



Policy Gradient Theorem

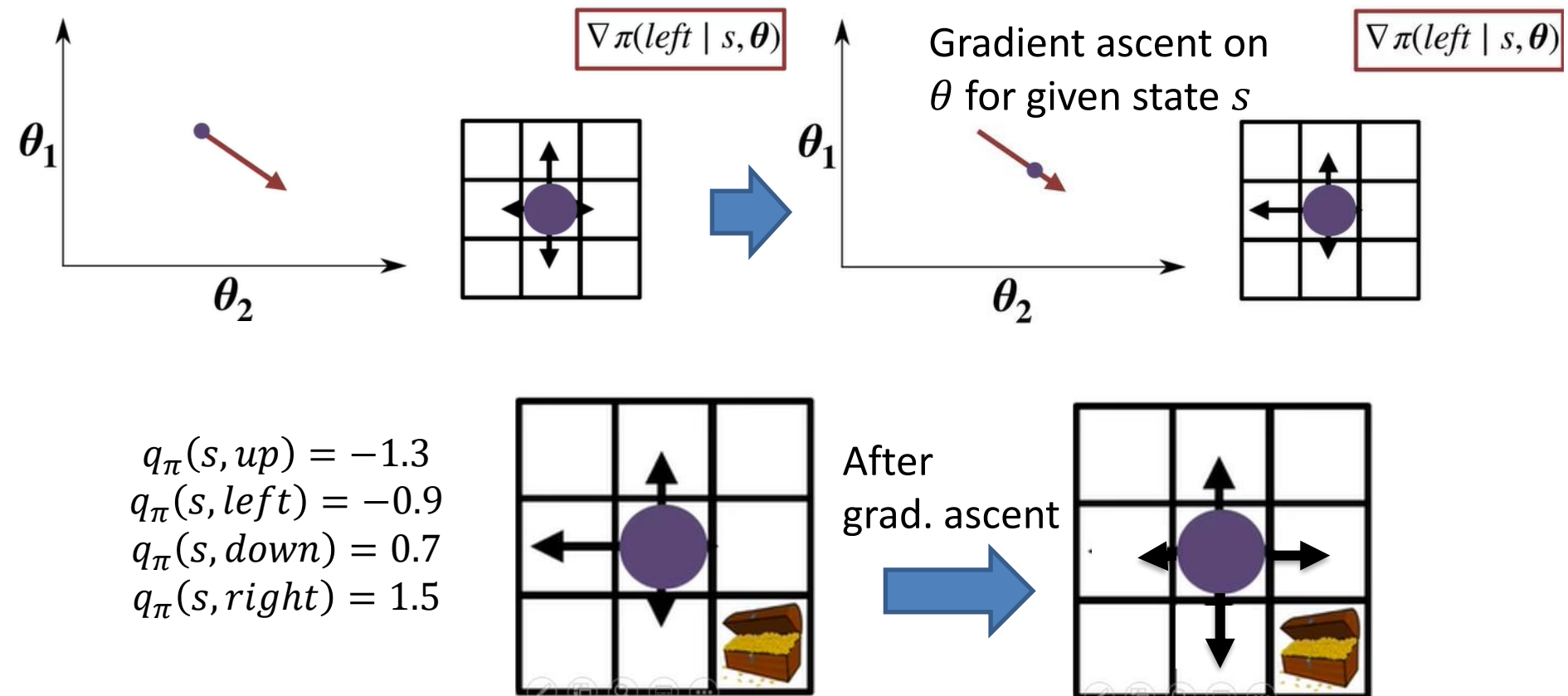
- We consider episodic instead of continuous environments in this lecture
- $\nabla J(\boldsymbol{\theta}) = \nabla v_{\pi}(s_0)$
- $= \nabla [\sum_a \pi(a|s, \boldsymbol{\theta}) q_{\pi}(s, a)]$
- $= \sum_a [\nabla \pi(a|s, \boldsymbol{\theta}) q_{\pi}(s, a) + \pi(a|s, \boldsymbol{\theta}) \nabla q_{\pi}(s, a)]$
- Policy Gradient Theorem (proof in RLBook p. 325; assuming discount factor $\gamma = 1$):
- $\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \nabla \pi(a|s, \boldsymbol{\theta}) q_{\pi}(s, a)$
 - $\mu(s)$: on-policy distribution under policy π , $\sum_{s \in \mathcal{S}} \mu(s) = 1$. A larger $\mu(s)$ denotes state s is visited more frequently, hence the policy gradient term for state s is given more weight (i.e., we care more about frequently-visited states than rarely-visited states).
 - In the episodic case, the constant of proportionality is the average length of an episode; in the continuing case it is 1.

Policy Gradient with Baseline

- $\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \nabla \pi(a|s, \boldsymbol{\theta})(q_\pi(s, a) - b(s))$
 - Baseline $b(s)$ can be any function that is independent of action a . Subtracting $b(s)$ does not affect the computed expectation since:
$$\sum_a \nabla \pi(a|s, \boldsymbol{\theta}) b(s) = b(s) \sum_a \nabla \pi(a|s, \boldsymbol{\theta}) = b(s) \nabla \sum_a \pi(a|s, \boldsymbol{\theta}) = b(s) \nabla 1 = 0$$
 - Subtracting $b(s)$ helps to reduce variance and speed up convergence, e.g., consider 3 samples with $\nabla \pi(a|s, \boldsymbol{\theta}) = \{0.5, 0.2, 0.3\}$, $q_\pi(s, a) = \{1000, 1001, 1002\}$, $\text{var}(0.5 \cdot 1000, 0.2 \cdot 1001, 0.3 \cdot 1002) \approx 23287$; After subtracting a baseline of 1001, $\text{var}(0.5 \cdot (-1), 0.2 \cdot 0, 0.3 \cdot 1) \approx 0.16$
- Use the state value function $v_\pi(s)$ as baseline $b(s)$:
 - Advantage function $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$ captures the advantage of taking action a in state s then follow policy π , compared to the baseline of following policy π from state s , i.e., we care more about the relative ranking of different actions in state S_t than absolute values of $q_\pi(s, a)$
 - $v_\pi(s)$ can be modeled as a NN that is learned/updated concurrently with the policy

Policy Gradient Example

- The gradient $\nabla \pi(\text{left} | s, \theta)$ tells us how to change the policy parameters θ to make action *left* more likely to be selected in state s . By gradient ascent on θ , we increase the probability for taking action *left* in state s .



MC REINFORCE

- $\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \nabla \pi(a|s, \boldsymbol{\theta}) q_\pi(s, a)$
- $= \mathbb{E}_{S_t \sim \mu(s)} [\sum_a \nabla \pi(a|S_t, \boldsymbol{\theta}) q_\pi(S_t, a)]$
 - Outer exp: average over all experienced states under policy π , i.e., $S_t \sim \mu(s)$
- $= \mathbb{E}_{S_t \sim \mu(s)} \left[\sum_a \pi(a|S_t, \boldsymbol{\theta}) \frac{\nabla \pi(a|S_t, \boldsymbol{\theta})}{\pi(a|S_t, \boldsymbol{\theta})} q_\pi(S_t, a) \right]$
- $= \mathbb{E}_{S_t \sim \mu(s)} [\sum_a \pi(a|S_t, \boldsymbol{\theta}) \nabla \log \pi(A_t|S_t, \boldsymbol{\theta}) q_\pi(S_t, a)]$ (since $\nabla \log f(x) = \frac{\nabla f(x)}{f(x)}$)
- $= \mathbb{E}_{S_t \sim \mu(s)} [\mathbb{E}_{a \sim \pi(a|S_t, \boldsymbol{\theta})} [\nabla \log \pi(A_t|S_t, \boldsymbol{\theta}) q_\pi(S_t, A_t)]]$
 - Inner exp over policy π : average over all experienced actions A_t from state S_t under policy π , i.e., $A_t \sim \pi(a|S_t, \boldsymbol{\theta})$
- $= \mathbb{E}_\pi [\nabla \log \pi(A_t|S_t, \boldsymbol{\theta}) q_\pi(S_t, A_t)]$
 - $\mathbb{E}_\pi$ as shorthand for $\mathbb{E}_{S_t \sim \mu(s)} \mathbb{E}_{a \sim \pi(a|S_t, \boldsymbol{\theta})}$: execute policy π and average over all experienced states S_t and actions A_t from state S_t
- $= \mathbb{E}_\pi [\nabla \log \pi(A_t|S_t, \boldsymbol{\theta}) G_t]$
 - Use return $G_t \doteq \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$ as unbiased estimate of $q_\pi(S_t, A_t)$ ($\mathbb{E}_\pi[G_t|S_t, A_t] = q_\pi(S_t, A_t)$)
 - $\nabla \log \pi(A_t|S_t, \boldsymbol{\theta})$ is called the **score function**
- SGD update ($\gamma = 1$): $\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t + \alpha \nabla \log \pi(A_t|S_t, \boldsymbol{\theta}_t) G_t$
- SGD update ($\gamma < 1$): $\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t + \alpha \gamma^t \nabla \log \pi(A_t|S_t, \boldsymbol{\theta}_t) G_t$

MC REINFORCE Example

- Consider a given state S with two possible actions **UP** and **DOWN**. Initially $\pi(U|S, \theta_t) = .9, \pi(D|S, \theta_t) = .1$. Consider one episode with 9 occurrences of (S, U) at steps $t1, t2, \dots, t9$ with return $G_{t1} = \dots = G_{t9} = 1$, and 1 occurrence of (S, D) at step $t10$ with return $G_{t10} = 2$. SGD update: $\theta_{t+1} \leftarrow \theta_t + \nabla \log \pi(A_t|S_t, \theta_t) G_t$. At the end of the episode, the parameters are updated as follows:
- Correct update sequence with $\nabla \log \pi = \frac{1}{\pi} \nabla \pi$:
 - 1st update at step $t1$: $\theta_{t1+1} = \theta_{t1} + \alpha \gamma^{t1} \nabla \log \pi(A_{t1}|S_{t1}, \theta_{t1}) G_{t1} = \theta_{t1} + \frac{1}{.9} \alpha \gamma^{t1} \nabla \pi(U|S, \theta_{t1}) \cdot 1$
 - 2nd update at step $t2$: $\theta_{t2+1} = \theta_{t2} + \alpha \gamma^{t2} \nabla \log \pi(A_{t2}|S_{t2}, \theta_{t2}) G_{t2} = \theta_{t2} + \frac{1}{.9} \alpha \gamma^{t2} \nabla \pi(U|S, \theta_{t2}) \cdot 1$
 - ...
 - 9th update at step $t9$: $\theta_{t9+1} = \theta_{t9} + \alpha \gamma^{t9} \nabla \log \pi(A_{t9}|S_{t9}, \theta_{t9}) G_{t9} = \theta_{t9} + \frac{1}{.9} \alpha \gamma^{t9} \nabla \pi(U|S, \theta_{t9}) \cdot 1$
 - 10th update at step $t10$: $\theta_{t10+1} = \theta_{t10} + \alpha \gamma^{t10} \nabla \log \pi(A_{t10}|S_{t10}, \theta_{t10}) G_{t10} = \theta_{t10} + \frac{1}{.1} \alpha \gamma^{t10} \nabla \pi(D|S, \theta_{t10}) \cdot 2$
- Incorrect update sequence with $\nabla \pi$:
 - 1st update at step $t1$: $\theta_{t1+1} = \theta_{t1} + \alpha \gamma^{t1} \nabla \log \pi(A_{t1}|S_{t1}, \theta_{t1}) G_{t1} = \theta_{t1} + \alpha \gamma^{t1} \nabla \pi(U|S, \theta_{t1}) \cdot 1$
 - 2nd update at step $t2$: $\theta_{t2+1} = \theta_{t2} + \alpha \gamma^{t2} \nabla \log \pi(A_{t2}|S_{t2}, \theta_{t2}) G_{t2} = \theta_{t2} + \alpha \gamma^{t2} \nabla \pi(U|S, \theta_{t2}) \cdot 1$
 - ...
 - 9th update at step $t9$: $\theta_{t9+1} = \theta_{t9} + \alpha \gamma^{t9} \nabla \log \pi(A_{t9}|S_{t9}, \theta_{t9}) G_{t9} = \theta_{t9} + \alpha \gamma^{t9} \nabla \pi(U|S, \theta_{t9}) \cdot 1$
 - 10th update at step $t10$: $\theta_{t10+1} = \theta_{t10} + \alpha \gamma^{t10} \nabla \log \pi(A_{t10}|S_{t10}, \theta_{t10}) G_{t10} = \theta_{t10} + \alpha \gamma^{t10} \nabla \pi(D|S, \theta_{t10}) \cdot 2$
- The correct update sequence gives *roughly equal* weight to the two action choices with return of 1 (action U in state S , experienced 9 times) and return of 2 (action D in state S , experienced once), so θ will be updated towards preferring action D in state S
- The incorrect update sequence gives *roughly 9 times* the weight to the action choice with return of 1 (action U in state S , experienced 9 times) than the action choice with return of 2 (action D in state S , experienced once), so θ will be incorrectly updated towards preferring action U in state S
- “The update increases the parameter vector in this direction proportional to the return, and inversely proportional to the action probability. The former makes sense because it causes the parameter to move most in the directions that favor actions that yield the highest return. The latter makes sense because otherwise actions that are selected frequently are at an advantage (the updates will be more often in their direction) and might win out even if they do not yield the highest return.” – RLBook p. 327

MC REINFORCE Pseudo-Code

- At the end of each episode, for each timestep $0 \leq t < T$:
 - Calculate the return G_t from each timestep t
 - Update policy params θ with SGD
 - If $G > 0$, update θ to increase $\pi(A_t|S_t, \theta)$ (encourage action A_t in state S_t)
 - If $G < 0$, update θ to decrease $\pi(A_t|S_t, \theta)$ (discourage action A_t in state S_t)
- Generate new samples (new episode) after each policy gradient update

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$\begin{aligned} G &\leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \\ \theta &\leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta) \end{aligned} \quad (G_t)$$

MC REINFORCE with Baseline

- Use the estimated state value function $\hat{v}_{\pi}(s, \mathbf{w})$ as baseline, whose params $\mathbf{w}$ are learned with a Monte Carlo method concurrently with the policy params θ

REINFORCE with Baseline (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Algorithm parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

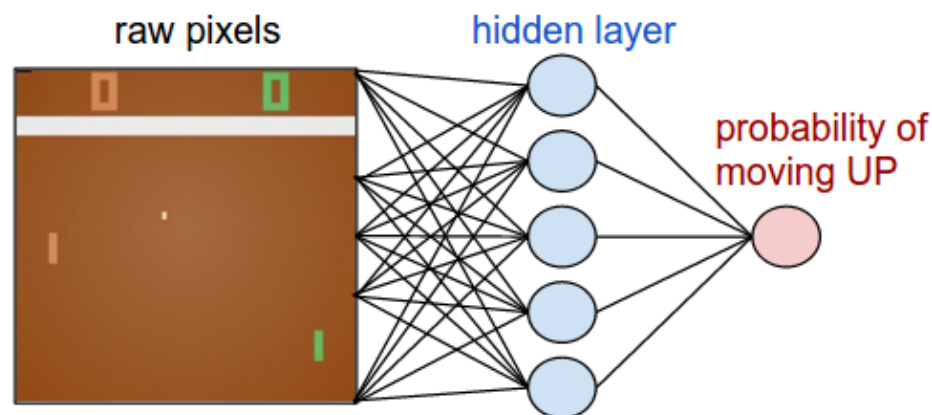
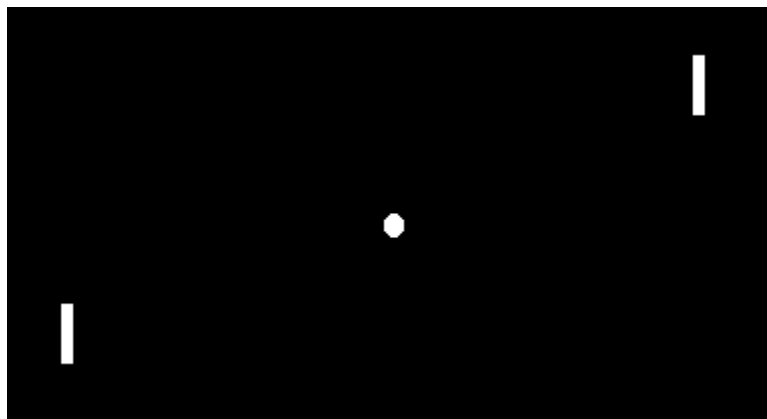
$$\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w})$$

$$\theta \leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t | S_t, \theta)$$

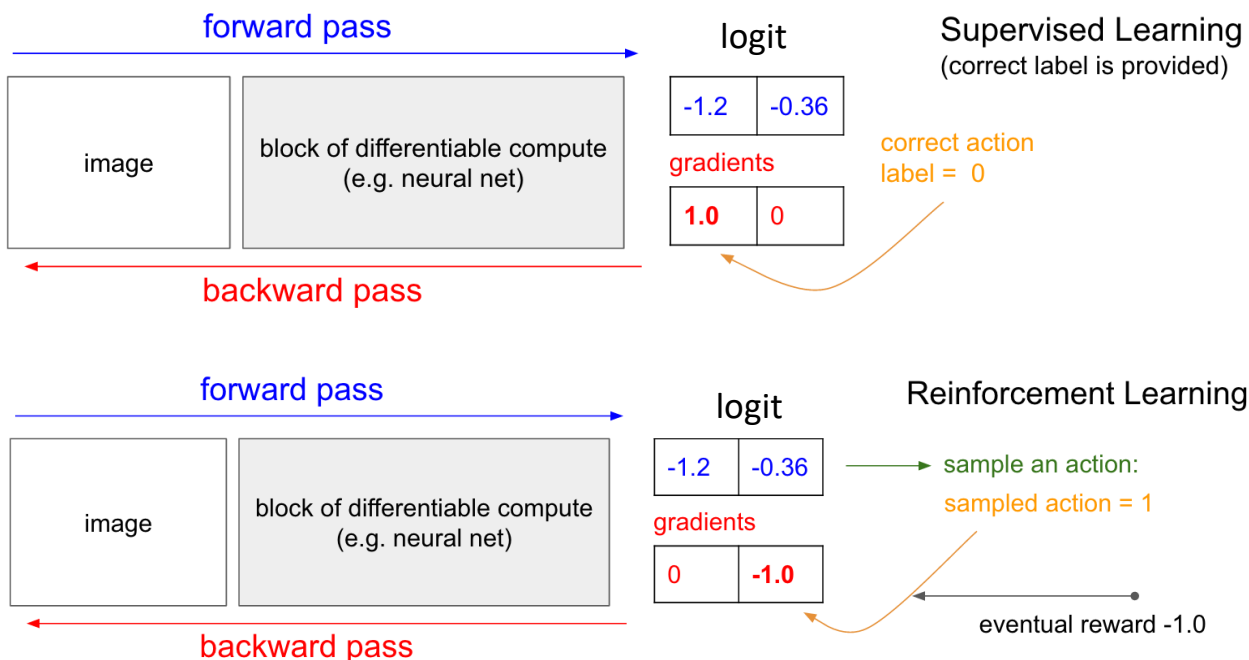
Example: Game of Pong

- The agent plays one of the paddles (the other is controlled by a decent AI) and it has to bounce the ball past the other player. For each input image, agent decides if it wants to move the paddle **UP** or **DOWN** (2 discrete actions). At the end of the game the agent either wins (reward $R_T > 0$) or loses ($R_T < 0$), i.e., the reward is sparse.
 - (In practice we may stack multiple input images as input to the agent.)
- The agent implements a policy network, which maps from input image to two possible actions (U or D) with a stochastic SoftMax policy.



Supervised vs. Reinforcement Learning

- With SL: if the model predicts $y = U$ for input x , and it is the correct ground truth label (e.g., the expert action in Imitation Learning), then gradient descent ($\nabla_{\theta} \log p(y = U|x)$) will make the model more likely to predict $y = U$ for input x
- With RL: if the model predicts $a = D$ for input x , but we don't have the ground truth label in the middle of the episode, so we must wait until the end of the episode. If agent loses, then gradient descent ($\nabla_{\theta} \log p(a = D|x)$) will make the NN less likely to predict $a = D$ for input x
 - Credit assignment problem: in an episode of many steps, which step contributed the most to the final outcome?



MC REINFORCE for Pong I

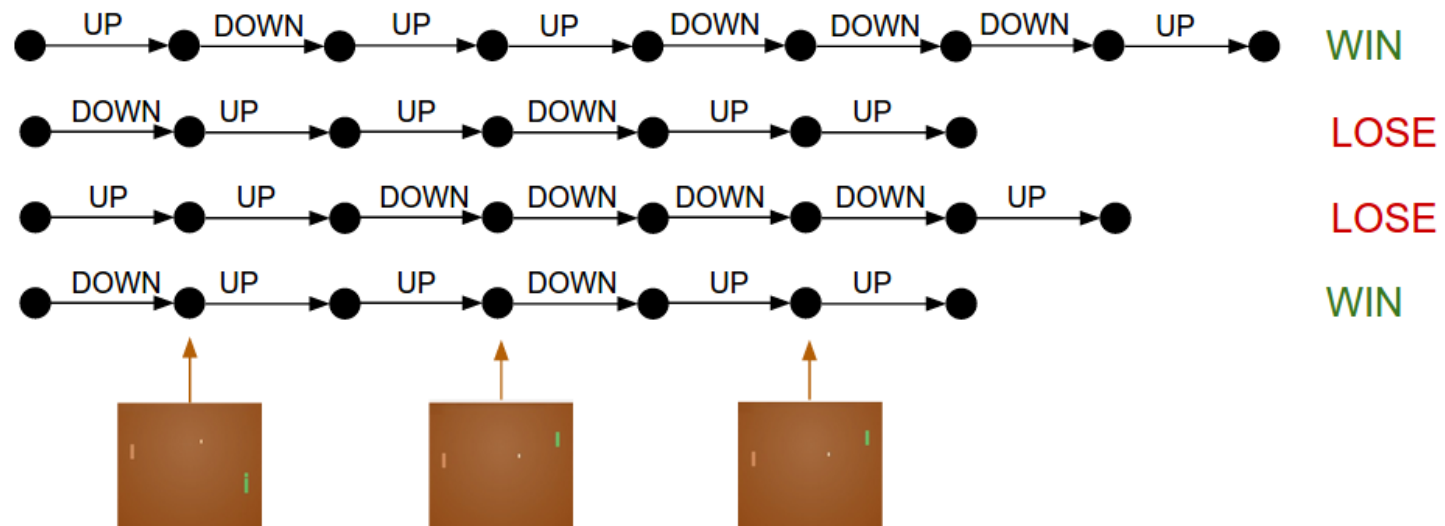
- $J(\theta) = \sum_s \mu_\pi(s) \sum_a \pi(a|s, \theta) q_\pi(s, a)$
- Consider agent in a given state S_t at time step t , and we want to select policy params θ to maximize
- $v_\pi(S_t) = \sum_a \pi(a|S_t, \theta) q_\pi(S_t, a)$
- $= \pi(U|S_t, \theta) q_\pi(S_t, U) + \pi(D|S_t, \theta) q_\pi(S_t, D)$
- Taking derivative w.r.t policy params θ
- $\nabla_\theta v_\pi(S_t) = \sum_a \nabla_\theta \pi(a|S_t, \theta) q_\pi(S_t, a)$
- $= \nabla_\theta \pi(U|S_t, \theta) q_\pi(S_t, U) + \nabla_\theta \pi(D|S_t, \theta) q_\pi(S_t, D)$
- $= \pi(U|S_t, \theta) \nabla_\theta \log \pi(U|S_t, \theta) q_\pi(S_t, U) + \pi(D|S_t, \theta) \nabla_\theta \log \pi(D|S_t, \theta) q_\pi(S_t, D)$
- $= \mathbb{E}_\pi [\nabla_\theta \log \pi(a|S_t, \theta) q_\pi(S_t, a)]$
- $= \mathbb{E}_\pi [\nabla_\theta \log \pi(a|S_t, \theta) G_t]$
- Suppose $q_\pi(S_t, U) > 0, q_\pi(S_t, D) < 0$.
 - For the UP action in state S_t , the policy update $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi(U|S_t, \theta) q_\pi(S_t, U)$ will push up $\pi(U|S_t, \theta)$, i.e, make it more likely to move UP in state S_t
 - For the DOWN action in state S_t , the policy update $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi(D|S_t, \theta) q_\pi(S_t, D)$ will push down $\pi(D|S_t, \theta)$, i.e, make it less likely to move DOWN in state S_t

MC REINFORCE

$$\begin{aligned}
 \nabla J(\theta) &\propto \sum_s \mu(s) \sum_a \nabla \pi(a|s, \theta) q_\pi(s, a) \\
 &= \mathbb{E}_\pi [\sum_a \nabla \pi(a|S_t, \theta) q_\pi(S_t, a)] \\
 &\quad - \text{Outer exp over policy } \pi: \text{average over all experienced states under policy } \pi, \text{ i.e., } S_t \sim s \\
 &= \mathbb{E}_\pi \left[\sum_a \pi(a|S_t, \theta) \frac{\nabla \pi(a|S_t, \theta)}{\pi(a|S_t, \theta)} q_\pi(S_t, a) \right] \\
 &= \mathbb{E}_\pi [\sum_a \pi(a|S_t, \theta) \nabla \log \pi(a|S_t, \theta) q_\pi(S_t, a)] \quad (\text{since } \nabla \log f(x) = \frac{\nabla f(x)}{f(x)}) \\
 &= \mathbb{E}_\pi [\mathbb{E}_\pi [\nabla \log \pi(A_t|S_t, \theta) q_\pi(S_t, A_t)]] \\
 &\quad - \text{Inner exp over policy } \pi: \text{average over all experienced actions } A_t \text{ from state } S_t \text{ under policy } \pi, \text{ i.e., } A_t \sim \pi(a|S_t, \theta) \\
 &= \mathbb{E}_\pi [\nabla \log \pi(A_t|S_t, \theta) q_\pi(S_t, A_t)] \\
 &\quad - \text{Outer and inner exp over policy } \pi \text{ combined: execute policy } \pi \text{ and average over all experienced states } S_t \text{ and actions } A_t \text{ from state } S_t, \text{ i.e., } S_t \sim s, A_t \sim \pi(a|S_t, \theta) \\
 &= \mathbb{E}_\pi [\nabla \log \pi(A_t|S_t, \theta) G_t] \\
 &\quad - \text{Use return } G_t \doteq \sum_{k=0}^{T-1} \gamma^k R_{t+k+1} \text{ as unbiased estimate of } q_\pi(S_t, A_t) (\mathbb{E}_\pi [G_t|S_t, A_t] = q_\pi(S_t, A_t)) \\
 &\quad - \nabla \log \pi(A_t|S_t, \theta) \text{ is called the } \text{score function} \\
 \text{SGD update } (\gamma = 1): \theta_{t+1} &\leftarrow \theta_t + \alpha \nabla \log \pi(A_t|S_t, \theta_t) G_t \\
 \text{SGD update } (\gamma < 1): \theta_{t+1} &\leftarrow \theta_t + \alpha \gamma^t \nabla \log \pi(A_t|S_t, \theta_t) G_t \quad (\text{proof omitted})
 \end{aligned}$$

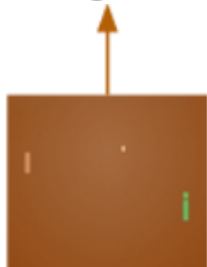
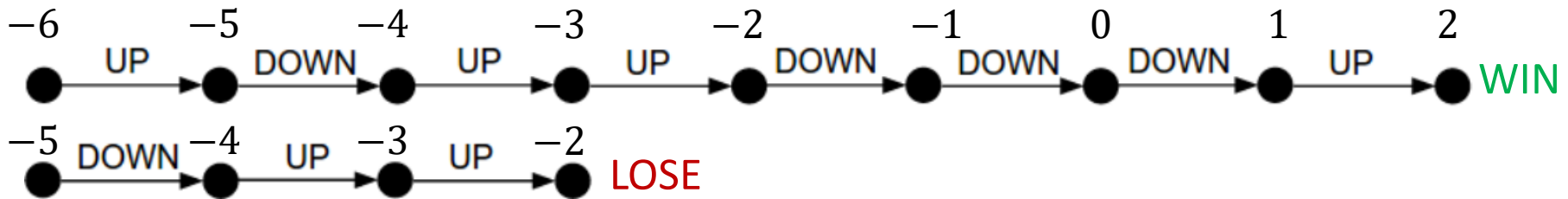
MC REINFORCE for Pong II

- Agent plays 4 rollouts (episodes), and won 2 episodes and lost 2. Assume that each episode lasts 200 steps, so agent made 200 decisions of UP or DOWN in each episode. Each step has reward of $R_t = 0$, i.e., we don't care how long each episode lasts. $\forall t, G_t = \gamma^{T-t-1} R_T$, i.e., return G_t at any step t of each episode is equal to the discounted reward R_T at the end of the episode, hence G_t has the same sign as R_T (recall “MC Prediction” in L7.2 Value-based RL).
- For each of the 2 **winning** episodes ($\forall t, G_t > 0$), NN params $\theta \leftarrow \theta + \alpha \nabla \log \pi(A_t | S_t, \theta) G_t$ are updated to **encourage** all taken actions in the 200 steps (**push up** $\pi(A_t | S_t, \theta)$).
- For each of the 2 **losing** episodes ($\forall t, G_t < 0$), NN params $\theta \leftarrow \theta + \alpha \nabla \log \pi(A_t | S_t, \theta) G_t$ are updated to **discourage** all taken actions in the 200 steps (**push down** $\pi(A_t | S_t, \theta)$).
- The NN will now become slightly more likely to repeat actions that worked, and slightly less likely to repeat actions that didn't work.
- If discount factor $\gamma = 1$, then we assign equal credit to all actions in the same episode; if $\gamma < 1$, then we assign more credit to actions taken in later steps of each episode than earlier steps.



MC REINFORCE for Pong III

- Consider the case when each step has reward of $R_t = -1$, to minimize the length of each episode, e.g., we may prefer a short losing episode to a long winning episode. Assuming discount factor $\gamma = 1$. Suppose reward at the end of each winning episode is $R_T = 2$; reward at the end of each losing episode is $R_T = -2$. The return G_t at each step is shown below. Consider state S_t .
- After the **winning** episode with return $-1 * 8 + 2 = -6$, params $\theta \leftarrow \theta + \alpha \nabla \log \pi(UP|S_t, \theta)(-6)$ are updated to **discourage** the U action in state S_t (**push down** $\pi(U|S_t, \theta)$).
- After the **losing** episode with return $-1 * 3 - 2 = -5$, params $\theta \leftarrow \theta + \alpha \nabla \log \pi(DOWN|S_t, \theta)(-5)$ are updated to **discourage** the D action in state S_t (**push down** $\pi(D|S_t, \theta)$).
- Both will push down $\pi(D|S_t, \theta)$, but since the return $G_t = -6$ in the winning episode causes a larger update magnitude than $G_t = -5$ in the losing episode, and the probabilities of all actions sum to 1 $\sum_a \pi(a|S_t, \theta) = 1$, the agent is slightly more likely to select the D action in state S_t .
- Subtracting a baseline, e.g., use $G_t - \hat{v}_\pi(S_t, \mathbf{w})$ to replace G_t helps to reduce variance and speed up convergence, e.g., suppose $\hat{v}_\pi(S_t, \mathbf{w}) = -5.5$, then updates to θ will be in different directions for the two episodes (**push down** $\pi(U|S_t, \theta)$ after the **winning** episode, and **push up** $\pi(U|S_t, \theta)$ after the **losing** episode).



State S_t

PG Variants

- MC REINFORCE (no bias, high variance):
 - $\nabla J(\boldsymbol{\theta}) = \mathbb{E}_{\pi}[\nabla \log \pi(A_t|S_t, \boldsymbol{\theta}) q_{\pi}(S_t, A_t)] = \mathbb{E}_{\pi}[\log \pi(A_t|S_t, \boldsymbol{\theta}) G_t]$
- MC REINFORCE with baseline of estimated state value function:
 - $\nabla J(\boldsymbol{\theta}) = \mathbb{E}_{\pi}[\log \pi(A_t|S_t, \boldsymbol{\theta}) (G_t - \hat{v}_{\pi}(S_t, \mathbf{w}))]$
 - $\hat{v}_{\pi}(S_t, \mathbf{w})$ is a function approximation of the true $v_{\pi}(S_t, \mathbf{w})$, e.g., a value network
- Actor-Critic (high bias, low variance)
 - $\nabla J(\boldsymbol{\theta}) = \mathbb{E}_{\pi}[\log \pi(A_t|S_t, \boldsymbol{\theta}) \delta_t]$
- Advantage Actor-Critic (A2C): Advantage function $\delta_t = R_{t+1} + \gamma \hat{v}_{\pi}(S_{t+1}, \mathbf{w}) - \hat{v}_{\pi}(S_t, \mathbf{w})$ (sample-based estimate of the advantage function $A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$)
- MC REINFORCE is a special case of A2C:
 - 1-step TD target (A2C): $R_{t+1} + \gamma \hat{v}_{\pi}(S_{t+1}, \mathbf{w})$
 - m -step TD target: $G_t^{\lambda} \doteq \sum_{k=0}^{m-1} \gamma^k R_{t+k+1} + \gamma^m \hat{v}_{\pi}(S_{t+m})$
 - MC target (MC REINFORCE): $G_t \doteq \sum_{k=0}^{T-1} \gamma^k R_{t+k+1}$

One-Step A2C Pseudo-Code

- For update to critic params $\mathbf{w}$, refer to L7.2 Value-based RL, p 75 “Semi-Gradient TD(0) for Estimating $\hat{v} \approx v_\pi$ ”

One-step Actor–Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^\theta > 0$, $\alpha^\mathbf{w} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Initialize S (first state of episode)

$I \leftarrow 1$

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

 Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^\mathbf{w} \delta \nabla \hat{v}(S, \mathbf{w})$

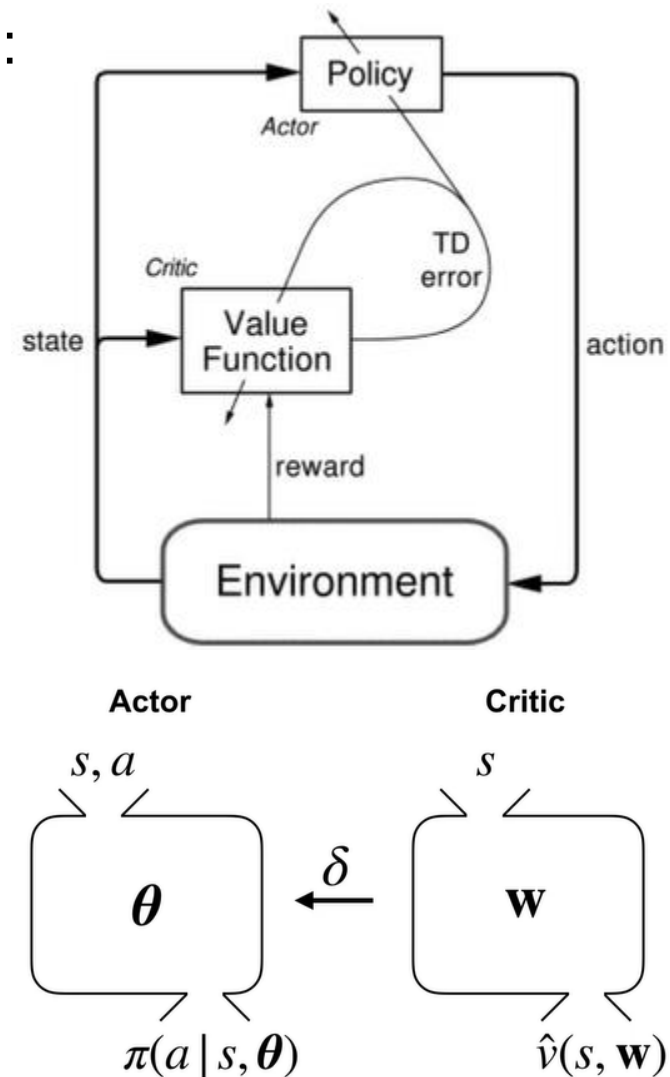
$\theta \leftarrow \theta + \alpha^\theta I \delta \nabla \ln \pi(A|S, \theta)$

$I \leftarrow \gamma I$

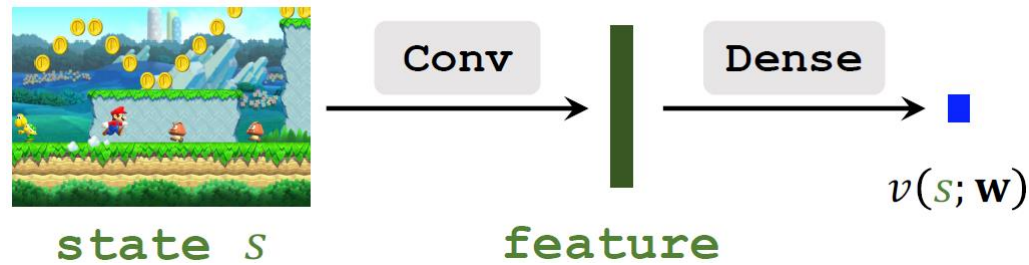
$S \leftarrow S'$

A2C Explanations

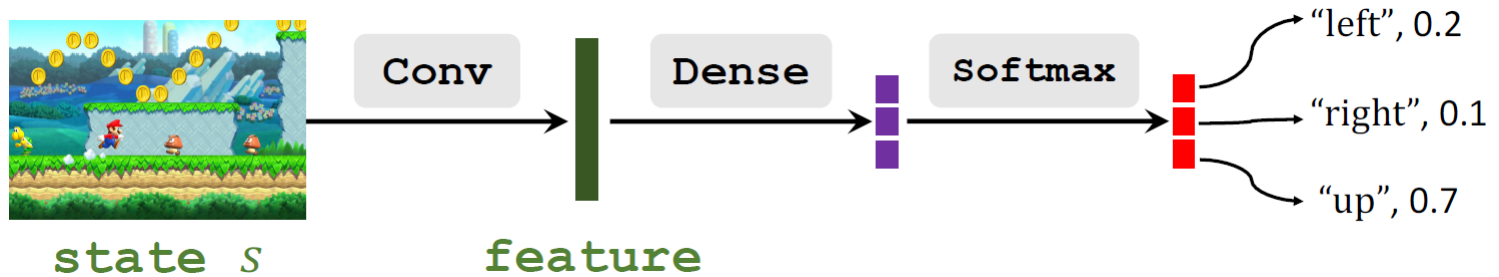
- After each step of taking action A_t in state S_t :
- Critic computes TD error $\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})$, and updates its params with semi-gradient TD(0) $\mathbf{w} \leftarrow \mathbf{w} + \alpha^w \delta_t \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$ (learning rate α^w)
- Actor updates its params with Policy Gradient $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha^\theta \gamma^t \delta_t \nabla_{\boldsymbol{\theta}} \log \pi(A_t | S_t, \boldsymbol{\theta}_t)$. If $\delta_t > 0$, then it means A_t resulted in a higher (one-step estimate) value than the expected $\hat{v}(S_t, \mathbf{w})$, so probability of A_t in state S_t is increased; if $\delta_t < 0$, it is decreased (learning rate α^θ)
- Actor and Critic learn at the same time, constantly interacting. The actor is continually changing the policy params $\boldsymbol{\theta}$ to exceed the critic's expectation, and the critic is constantly updating its value function params $\mathbf{w}$ to evaluate the actor's changing policy.



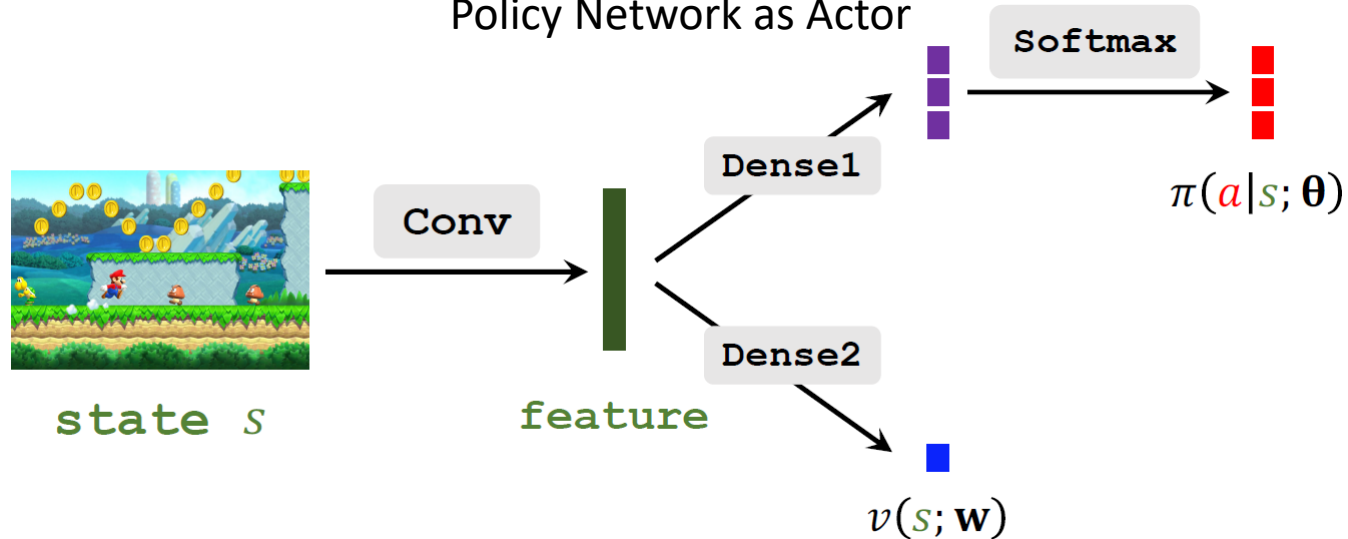
Function Approximations for Critic and Actor



Value Network as Critic



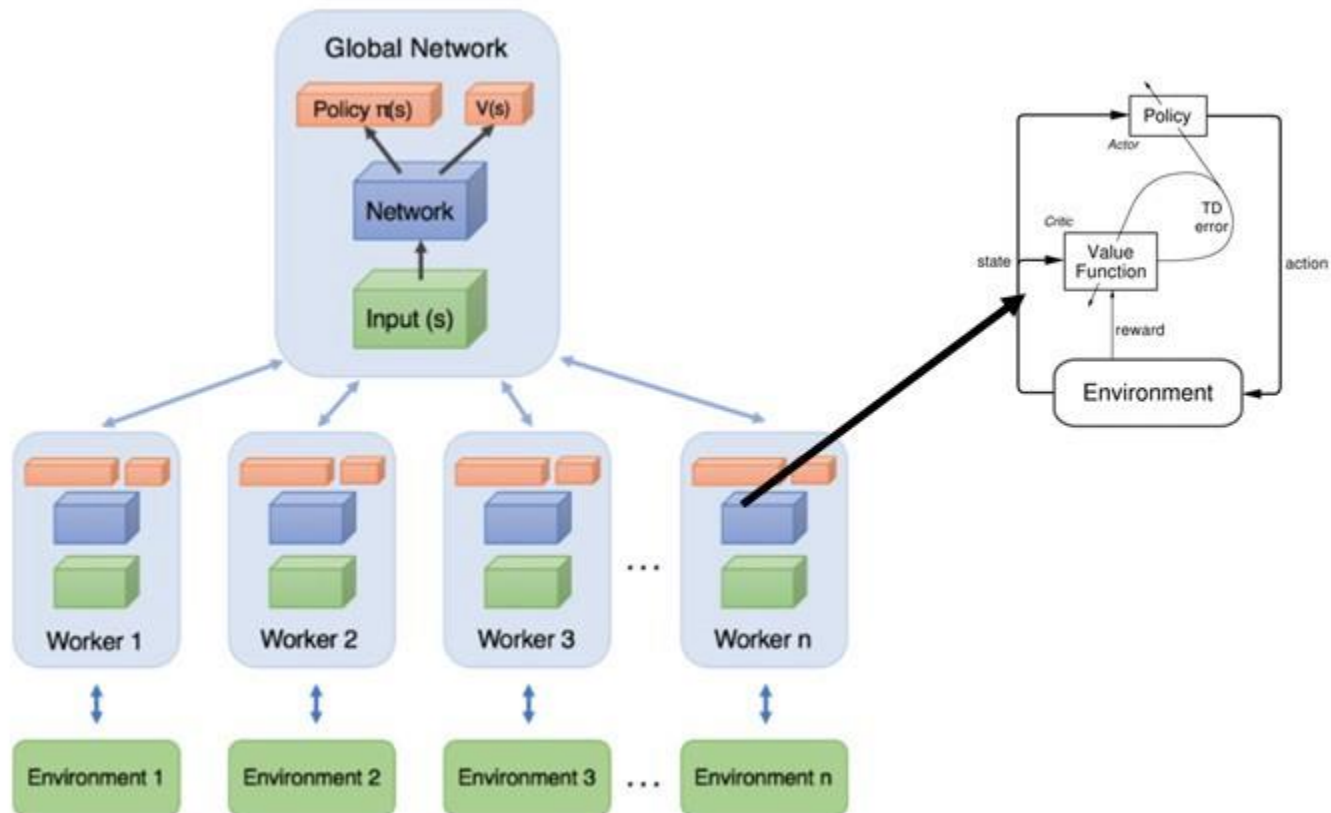
Policy Network as Actor



Parameter Sharing between Value and Policy Networks

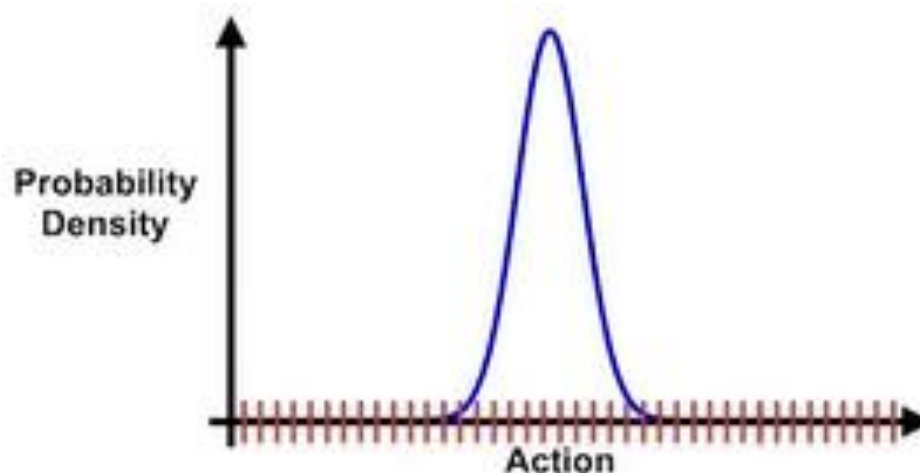
Asynchronous Advantage Actor Critic (A3C)

- A3C implements parallel training where multiple workers in parallel environments independently update the global value and policy networks, for effective and efficient exploration of the state space.



Continuous Actions

- It might not be straightforward to choose a proper discrete set of actions
- Continuous actions allow us to generalize over actions
 - If an action is good, its neighboring actions are also likely to be good
 - Discrete actions lack generalization: each action is independent of others, including its neighbors (similar to value functions for discrete states)

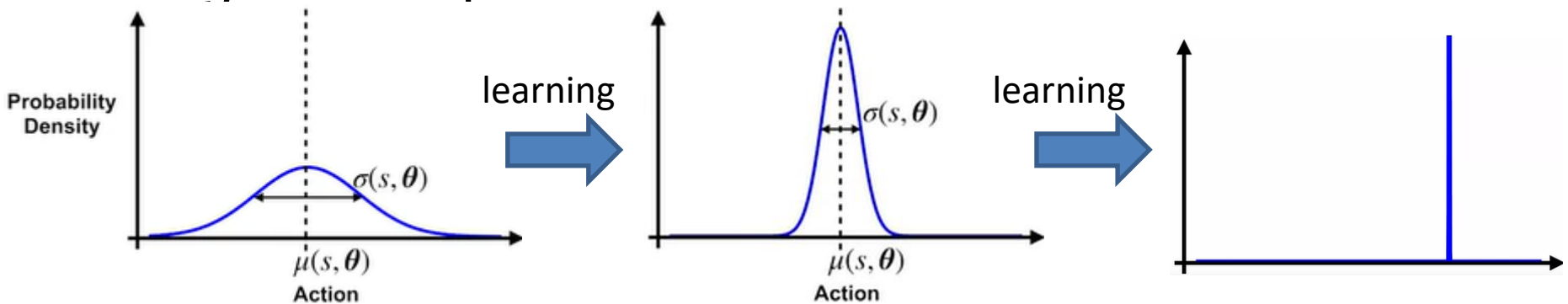
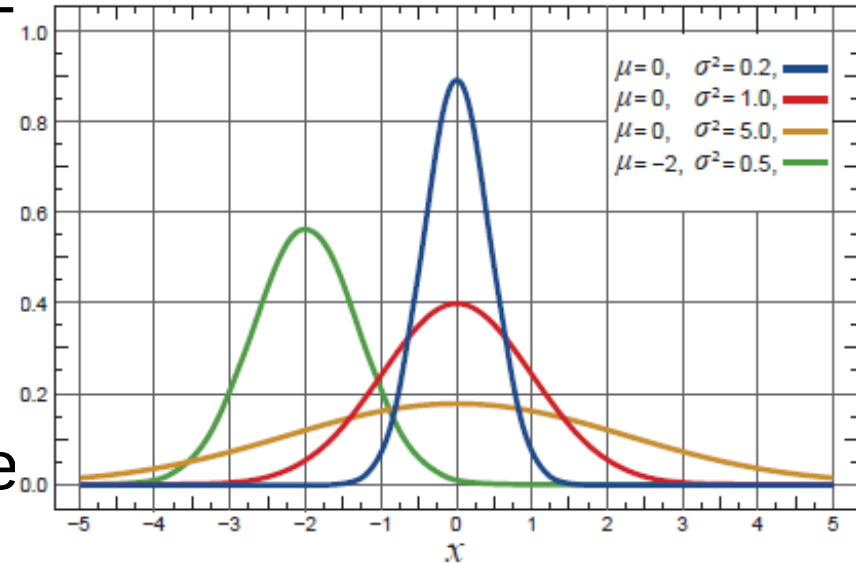


Gaussian Policy for Continuous Actions

- Gaussian Policy $\pi(a|s, \theta) \doteq$

$$\frac{1}{\sigma(s, \theta)\sqrt{2\pi}} \exp\left(-\frac{(a - \mu(s, \theta))^2}{2\sigma(s, \theta)^2}\right)$$

- Mean $\mu(s, \theta)$ is the most likely action
- Variance $\sigma(s, \theta)^2$ controls the degree of exploration.



Policy variance initially large,
more exploration

Variance gradually reduced during learning w. PG,
converging towards deterministic policy $a = \mu(s, \theta)$